Multi-Tenant Security Isolation

one missing WHERE clause away from disaster

Watch the 4-min video

In one line: pick an isolation model (silo, pool or bridge), take the tenant only from the authenticated identity, carry it into every query, cache key and limit, enforce it in the database with row-level security, and test for cross-tenant leaks in CI.

1 The worst bug you can ship

A SaaS serves 5,000 companies from one database. A developer forgets one `WHERE tenant_id = ?`, and company A sees company B's invoices. In multi-tenant systems, a cross-tenant leak is a breach of every customer's trust at once, and it takes only one missing line.

2 Three isolation models

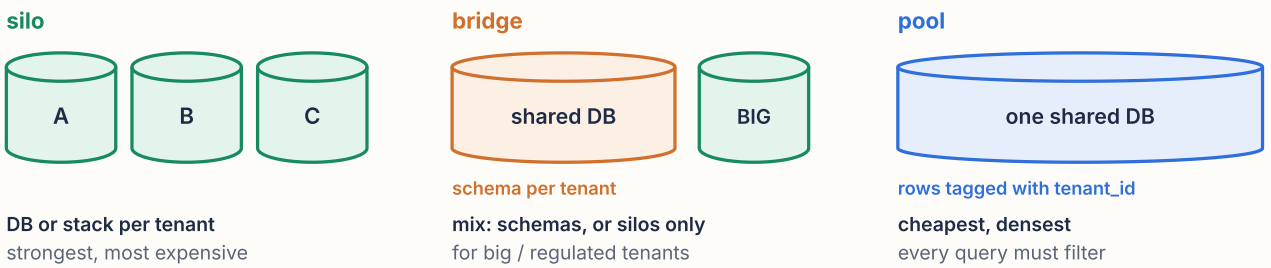


Figure 1. Silo, bridge and pool (the AWS SaaS Lens names). Most SaaS products pool small tenants and silo the largest.

	Silo	Bridge	Pool
Isolation	separate DB, account or cluster	schema per tenant, or mixed	shared tables + `tenant_id`
Cost, ops	highest; many deployments	medium; migrations per schema	lowest; one deployment
Leak risk	lowest	medium	highest; needs enforcement layers
Noisy neighbours	none	some	need quotas and limits

3 Tenant from identity, everywhere

In a pool, the tenant ID must come from the **authenticated identity** (a `tid` claim in the token, lesson 41), never from the request body, a header or a URL parameter the user controls. Resolve it once at the edge, pass it through every layer, and include it in every query, every cache key (`tenant:acme:invoice:1042`, or a shared cache serves one tenant's data to another), every queue message and every search filter.

4 Enforce it in the database

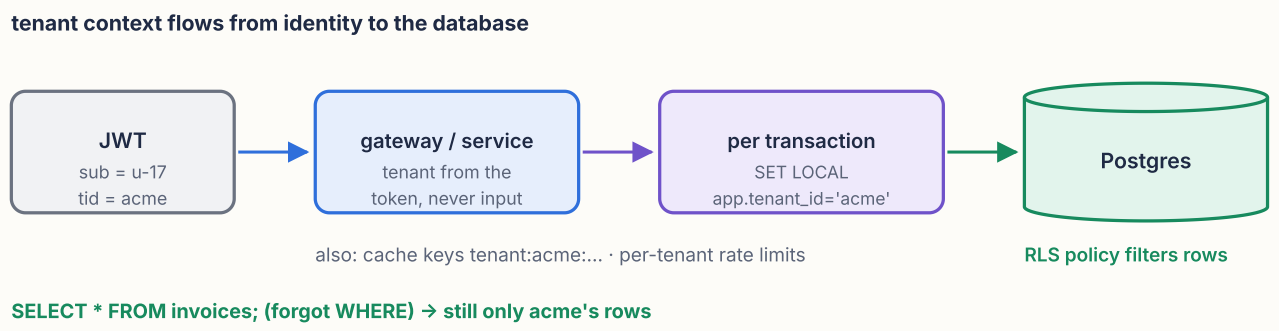


Figure 2. Row-level security makes the database apply the tenant filter, even when application code forgets it.

Don't rely on every developer remembering the filter. **Postgres row-level security** attaches a policy to each table; every query from the app's role is silently filtered. Two details matter: table owners and roles with `BYPASSRLS` skip policies (use `FORCE ROW LEVEL SECURITY` and a separate app role), and with connection pooling set the tenant with `SET LOCAL` inside each transaction so it can't leak to the next request.

5 Beyond rows

Per-tenant encryption keys (lesson 47) let you shred or lock one tenant independently, and support customer-managed keys. **Noisy neighbours**: one huge tenant can starve others of CPU, connections or queue capacity, so add per-tenant rate limits, quotas and fair scheduling. Large or regulated tenants may need dedicated infrastructure. And **test for leakage**: CI tests that log in as tenant A and request tenant B's resources by ID through every endpoint; alert on any cross-tenant access as a security event.

WORKED EXAMPLE: POSTGRES RLS ON EVERY TABLE

`ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;`
`CREATE POLICY tenant_isolation ON invoices USING (tenant_id = current_setting('app.tenant_id'));`
The app connects as a role with `NOBYPASSRLS` that doesn't own the tables, and runs `SET LOCAL app.tenant_id = …` at the start of each request's transaction. The policy's `USING` clause also checks inserted and updated rows, so no one can write into another tenant. A forgotten filter now returns nothing extra.

COMMON MISTAKES

- Reading `tenant_id` from a request parameter or header.
- Cache keys without the tenant, so one tenant's cached response is served to another.
- Running the app as the table owner or a superuser, which silently bypasses RLS.
- Background jobs and admin tools that skip the tenant context entirely.

6 Key terms

Tenant: a customer organisation sharing the system

Silo / pool / bridge: dedicated / shared / mixed isolation models

Row-level security: database policies that filter rows per session

Tenant context: tenant ID carried from identity through all layers

Noisy neighbour: one tenant's load degrading others

Cross-tenant leak: data from one tenant visible to another

7 Interview questions

Q1. How do you guarantee one tenant never sees another's data? BASIC

Take the tenant from the authenticated identity, include it in every query and cache key, enforce it in the database with row-level security, use per-tenant keys where needed, and run cross-tenant leakage tests in CI.

Q2. Which isolation model would you pick? BASIC

Pool (shared tables + tenant_id) for cost, silo for the strongest isolation, or a bridge: pool most tenants and silo the largest or regulated ones.

Q3. An enterprise customer demands their data never shares a database with others. What do you offer? SCENARIO

A silo tier: dedicated database (or account/cluster) with its own customer-managed key, deployed by the same automation, priced accordingly.

Q4. One tenant's nightly import slows everyone down. What do you do? SCENARIO

Per-tenant quotas and rate limits, separate queues or worker pools for bulk jobs, and move very large tenants to dedicated capacity.

Q5. RLS is enabled but a test still sees all tenants' rows. Why? TRICKY

The connection's role owns the table or has BYPASSRLS (or is a superuser); owners skip policies unless FORCE ROW LEVEL SECURITY is set.

Q6. Why SET LOCAL instead of SET for the tenant setting? TRICKY

With pooled connections, a plain SET persists on the connection and can leak into the next request; SET LOCAL lasts only until the transaction ends.

Next: SD-M-020 · Supply Chain Security backendroadmap.com